

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA	Total Marks:	100 (2 Questions × 50 Marks)
CO1 Statement:	<i>Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)</i>		
Student Name:	kavitha b	Reg. No.:	19255041
Section / Batch:		Date:	

Instructions to Students

- Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.
- Clearly show all steps in derivations and complexity analysis.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) wherever required.
- Justify each step in recurrence solving using appropriate methods.
- Neat presentation and logical explanation are mandatory

Question Type	Focus Area	Questions	Marks
Application-Based Questions	Apply Master Theorem and asymptotic analysis to real-world scenarios	Q1 – Q2	Q1 –50 Q2 –50
GRAND TOTAL:		100 Marks	

SECTION A – APPLICATION BASED QUESTIONS

Q1. Document Processing System

A document processing system processes files sequentially using the recurrence $T(n) = T(n-1) + 1$, where each step handles one document. Apply the substitution method to expand the recurrence relation step-by-step and derive the final time complexity. Clearly explain how the recurrence unfolds and simplifies into a closed-form expression. Analyze the space complexity, particularly in recursive implementations where function calls are stacked. Interpret how this linear approach affects scalability when processing a large number of documents.

iven the recurrence:

$$T(n)=T(n-1)+1$$

Substitution Method:

$$T(n)=T(n-1)+1=T(n-2)+2=T(n-3)+3:=T(1)+(n-1)$$

If $T(1)=1$,

$$T(n)=n$$

Time Complexity

Since one document is processed during each recursive call, the number of operations increases directly with the number of documents.

Time Complexity = $O(n)$

Space Complexity

Recursive Implementation

Each recursive call remains in memory until all recursive calls finish.

For **n** documents:

- Number of recursive calls = **n**
- Stack frames = **n**

Therefore,

Space Complexity = $O(n)$

Iterative Implementation

If the same algorithm is implemented using a loop instead of recursion:

- No recursive call stack
- Only a few variables are required

Therefore,

Space Complexity = $O(1)$

Scalability Analysis

- The processing time increases linearly as the number of documents increases.
- Doubling the number of documents approximately doubles the execution time.
- Recursive implementation may lead to stack overflow for very large datasets due to deep recursion.
- An iterative implementation is more memory-efficient and suitable for processing millions of documents.

Application Interpretation

This recurrence models many real-world systems where items are processed sequentially, such as:

- Document management systems
- File upload processing
- Email processing queues
- Batch printing systems
- Log file analysis
- Data migration task

Q2. Password Lookup System

A secure password lookup system uses a binary search technique to locate user credentials in a sorted database by repeatedly dividing the search space into halves. Define logarithmic time

complexity and explain how reducing the input size affects the number of operations performed. Determine the time complexity of the algorithm and justify it using asymptotic notation. Also analyze the space complexity, especially in recursive implementations of binary search. Finally, interpret how this method ensures efficient performance even for very large databases.

A secure password lookup system stores user credentials in **sorted order** and uses **binary search** to find a specific user. Instead of checking every record, binary search compares the target with the middle record. If the target is smaller, it searches the left half; otherwise, it searches the right half. This process continues until the password record is found or the search space becomes empty.

Logarithmic Time Complexity

In binary search, the search space is **reduced to half** after every comparison. Therefore, the number of operations increases very slowly, even when the database becomes very large.

Example:

- 16 records → at most **4** comparisons
- 32 records → at most **5** comparisons
- 64 records → at most **6** comparisons

Thus, doubling the number of records increases the search time by only **one extra comparison**.

Time Complexity

Since the search space is halved in every step,

Time Complexity = $O(\log n)$

where **n** is the number of records in the database.

Space Complexity

- **Recursive Binary Search:** Uses recursive function calls, so the call stack requires **$O(\log n)$** space.
- **Iterative Binary Search:** Uses only a few variables, so the space complexity is **$O(1)$** .

Application Interpretation

Binary search provides an efficient way to locate user credentials in a sorted password database. Because it eliminates half of the remaining records at each step, it performs far fewer comparisons than linear search. This makes it highly suitable for authentication systems with millions of user records, providing **fast search performance, reduced execution time, and efficient scalability**.

Final Result

- **Algorithm:** Binary Search
- **Time Complexity:** $O(\log n)$

- **Recursive Space Complexity:** $O(\log n)$
- **Iterative Space Complexity:** $O(1)$
- **Scalability:** Efficient for very large databases because the number of comparisons grows very slowly as the database size increases.

Code of Conduct

I _____ kavitha (192565041) _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Marks Evaluation:

Question No.	Understanding of Problem Context (20%)	Application of Concepts (30%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (10%)	Total Marks
Q1 (50 Marks)						
Q2 (50 Marks)						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____